1 Cross-Compilation Toolchain

1.1 Host, Target, and Emulator

Role	Meaning
Host	Computer running the editor, `make`, and the toolchain (x86-64 or Apple silicon)
Target	ARM Cortex-M3 machine for which the ELF is built
Emulator	QEMU model that executes target instructions on the host

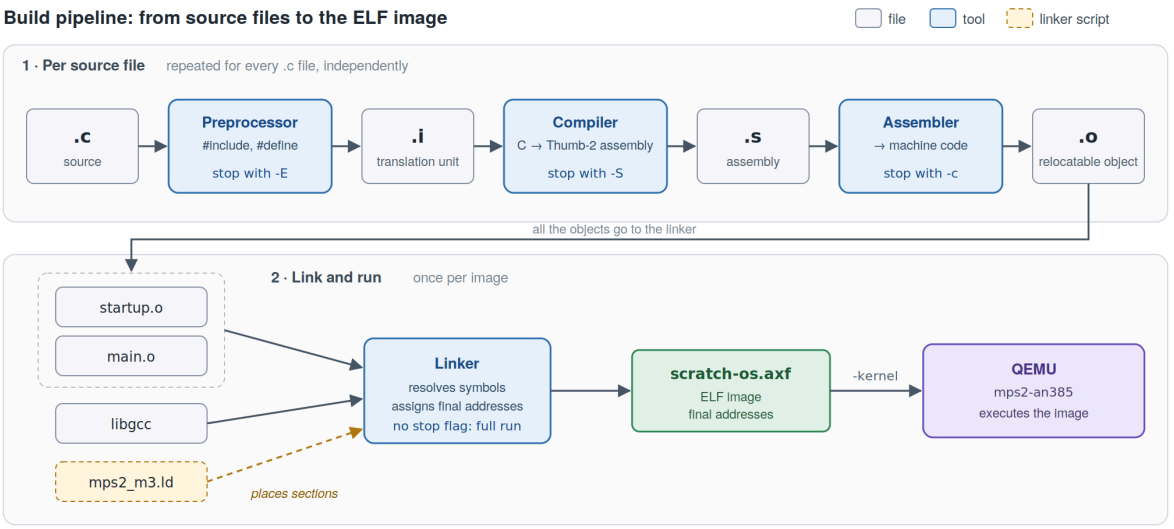
A native `gcc` emits code for the machine it runs on; it cannot produce Cortex-M3 firmware just because the source language is C. A **cross toolchain** (`arm-none-eabi-*`) runs on the host but emits code for the target instruction set architecture (ISA).

```
gcc -dumpmachine              # host triple, e.g. x86_64-linux-gnu
arm-none-eabi-gcc -dumpmachine  # target triple: arm-none-eabi
```

The triple `arm-none-eabi` means: ARM architecture, **no** operating system on the target, standard ARM **E**mbedded **ABI**.

- Consequences of "no OS":
  - No `_start` from a C library unless the application provides one.
  - No dynamic loader, and no process to return to when `main()` finishes.
  - No guaranteed `printf` console, no `malloc` backed by an OS.
- QEMU is not the target CPU, but it provides a reproducible board model before physical hardware is used.

1.2 Build Pipeline Stages



Stage	What it does	stop flag	ext
Preprocessing	Expands `#include`, macros, conditional compilation; pure text substitution	`-E`	`.i`

Stage	What it does	stop flag	ext
Compilation	Checks syntax and types, **optimises**, emits **ARM assembly** for the selected CPU and ABI	`-S`	`.s`
Assembly	Encodes assembly into a **relocatable object file**	`-c`	`.o`
Linking	Combines objects, resolves symbols, assigns final addresses	N/A	`.axf` / `.elf`

`arm-none-eabi-gcc` is a *driver*: a single invocation runs all four stages in sequence and shows **only the final result**, deleting the intermediate files. A stop flag interrupts the pipeline after a given stage and leaves its output on disk, so that the intermediate result can be inspected:

```
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -ffreestanding <stop flag> src/main.c -o build/main.<ext>
```

1.3 Target Compiler and Linker Flags

```
CFLAGS := -mthumb -mcpu=cortex-m3 -nostartfiles -ffreestanding -std=gnu17
CFLAGS += -Wall -Wextra -O0 -g3

LDFLAGS := -T scripts/mps2_m3.ld
LDFLAGS += -Xlinker -Map=build/output.map
LDFLAGS += -nostdlib -lgcc
```

Flag	Meaning
`-mthumb -mcpu=cortex-m3`	Selects the Thumb instruction subset implemented by the Cortex-M3
`-nostartfiles`	Excludes the toolchain's default C startup objects, so reset handling is done by the project's own startup code
`-ffreestanding`	Compiles for a freestanding (bare-metal) C environment, without hosted-library assumptions
`-std=gnu17`	C17 with GNU extensions
`-Wall -Wextra`	Warning policy
`-O0 -g3`	No optimisation and full debug information: a debuggable, source-corresponding image
`-T scripts/mps2_m3.ld`	Uses the given linker script to describe memory and section placement
`-Xlinker -Map=build/output.map`	Makes the linker emit a map file of every placement and symbol address
`-nostdlib -lgcc`	Links no standard library, only `libgcc` for compiler support routines

DIAGNOSING A BUILD FAILURE

The full command line is the exact contract that produced the failure: copy it before changing anything.

1.4 Separate Compilation and Linking

The compiler translates one source file at a time and never looks at the others. When `startup.c` calls `main()`, the compiler does not know where `main` is, or whether it exists: it emits the call and leaves its target address blank, to be filled in later. Connecting the files is the job of the linker.

1.4.1 Object Files and Symbols

The result of compiling is a **relocatable object file** (`.o`): machine code whose addresses are not yet final, plus the *symbols* (names of functions and global variables) that the file defines or only references. `arm-none-eabi-nm` lists the symbols of an object, one per line: address, a letter for the kind of symbol, and name. The meaning of each letter is listed in the chapter Inspecting the Image.

`STARTUP.O` REFERENCES `MAIN`, `MAIN.O` DEFINES IT

The two source files are compiled separately with `-c` .

```
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -ffreestanding -c boot/startup.c -o
build/boot/startup.o
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -ffreestanding -c src/main.c -o
build/src/main.o
```

Then their symbols are listed:

```
arm-none-eabi-nm build/boot/startup.o build/src/main.o
```

Output:

```
build/boot/startup.o:
00000008 T Reset_Handler
         U _estack
         U main
...
build/src/main.o:
00000018 T main
00000000 t semihost_write0
```

`startup.o` defines `Reset_Handler` but only references `main` and `_estack` (`U`); `main.o` defines `main` (`T`). The addresses are offsets inside each object, not final addresses: neither object is a runnable program on its own.

1.4.2 The Linker: Resolving Symbols

The **linker** combines all the objects into one image in three steps:

1. it concatenates the sections with the same name from every object, which gives each function and variable its final address;
2. it resolves every undefined symbol against a definition, here `main` against `main.o` and `_estack` against the linker script, and writes the final address into every instruction that referenced it;
3. it checks that every referenced symbol has exactly one definition.

> **`MAIN` AFTER THE LINK**
>
> The two objects are linked into the final image:
>
> ```
> arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -nostdlib \
>     -T scripts/mps2_m3.ld -Xlinker -Map=build/output.map \
>     build/boot/startup.o build/src/main.o -lgcc -o build/scratch-os.axf
> ```
>
> Then the symbols of the image are listed, sorted by address:
>
> ```
> arm-none-eabi-nm -n build/scratch-os.axf
> ```
>
> Output:
>
> ```
> 00000048 T Reset_Handler
> 000000a4 t semihost_write0
> 000000bc T main
> ...
> 20400000 A _estack
> ```
>
> The `U` lines have disappeared: `main` is now defined at its final address in Flash, and `_estack` is defined by the linker script (`A` , an absolute value that belongs to no section). Every address is the real one: `0x48` , `0xA4` , and `0xBC` match the Flash layout shown by the map file, which also records which object supplied each symbol.

Where the sections are placed is not decided by the linker on its own: the **linker script** (`.ld`) gives the addresses of `FLASH` and `RAM` and the order of the sections. On a desktop a default script is used without the developer seeing it; on **bare metal it must be supplied**, because only the developer knows the chip's memory map.

1.4.2.1 COMMON LINK ERRORS

The check of step 3 produces the **two most common link errors**:

Error	Cause
`undefined reference to 'main'`	A `U` that no object defines: a missing `.o` on the command line or a misspelled name

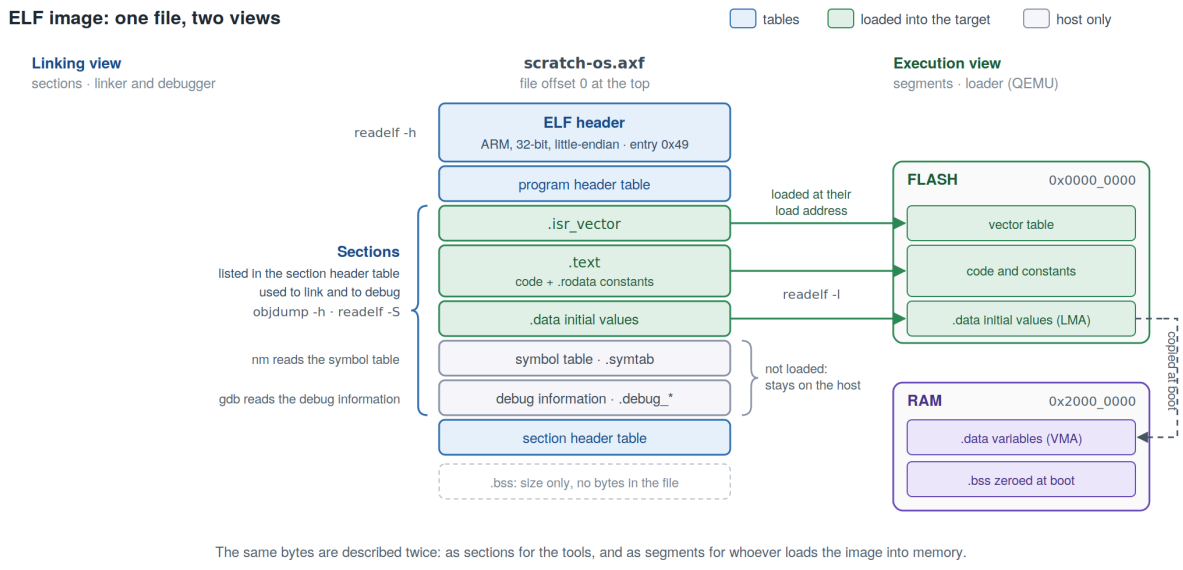
Error	Cause
`multiple definition of 'main'`	Two objects define the same symbol

2 The ELF Format

2.1 Structure of an ELF File

ELF (Executable and Linkable Format) is the format of the final image `build/scratch-os.axf`; the `.axf` suffix conventionally denotes an ARM executable with debug information. The image is a 32-bit, little-endian ARM ELF file that offers several views of the same bytes:

Part	Content	Used by
ELF header	At offset 0: file type, architecture, entry point, location of the other tables	Every tool that opens the file
Sections	`.text`, `.rodata`, `.data`, `.bss`, symbol table, debug information	Linker and debugger
Program headers (segments)	Which chunks of the file go to which memory addresses at run time	Loader (here QEMU)
Symbols	Names of functions and variables with their final addresses	Debugger, developer



The same bytes are described twice: as sections for the tools, and as segments for whoever loads the image into memory.

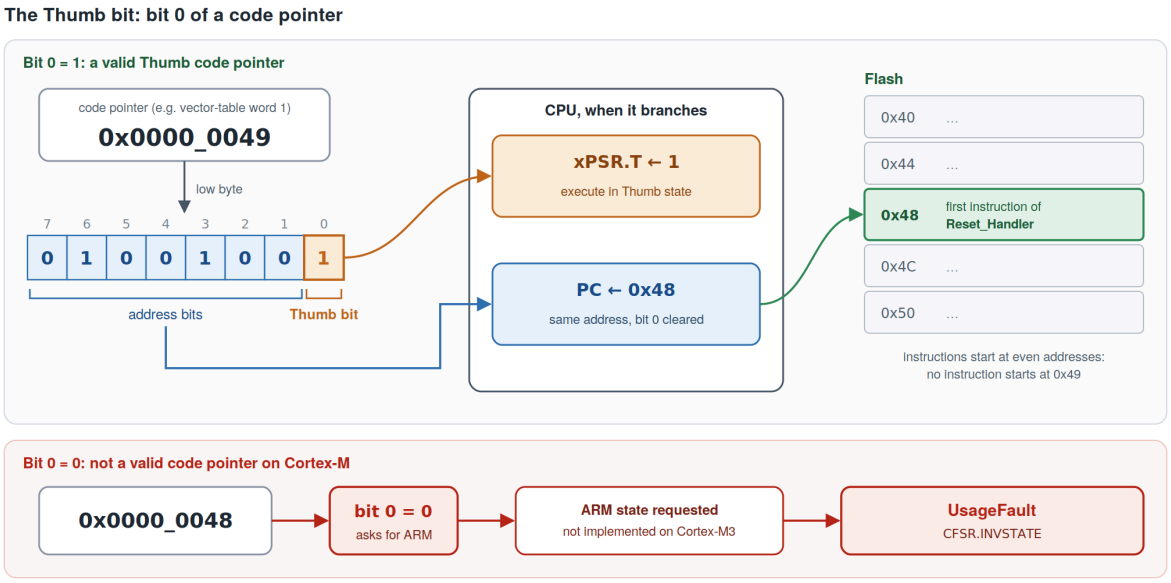
2.2 Entry Point and the Thumb Bit

ARM processors define two instruction sets:

- **ARM**, with 32-bit instructions,
- **Thumb**, with 16-bit instructions, extended in Thumb-2 to a mix of 16- and 32-bit ones. A core that supports both must know, at every branch, **which set the code at the target uses**.

> **THE CORTEX-M3 EXECUTES ONLY THUMB, BUT KEEPS THE SAME CONVENTION.**

Thumb instructions are 2 or 4 bytes long and always start at an **even address**, so bit 0 of an instruction address is always 0 and carries **no information**.



ARM reuses that bit as a flag, the *Thumb bit*:

- **Bit 0 = 1**: the target is **Thumb code**.
  - The CPU copies the bit into the `T` bit of `xPSR` (the special register that describe the current state of the processor), clears it, and jumps to the even address. The `T` bit of `xPSR` represents the Thumb State.

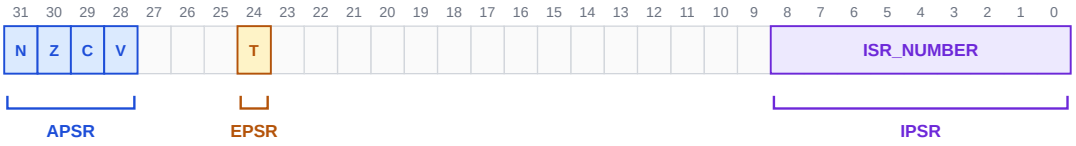


Figure 1.2: *Bit layout of xPSR: N, Z, C, V flags in APSR, Thumb bit in EPSR, exception number in IPSR.*

- **Bit 0 = 0**: the target would be ARM code, which the Cortex-M3 **does not implement**.
  - The branch raises a UsageFault (`INVSTATE`) instead of executing anything at the target address.

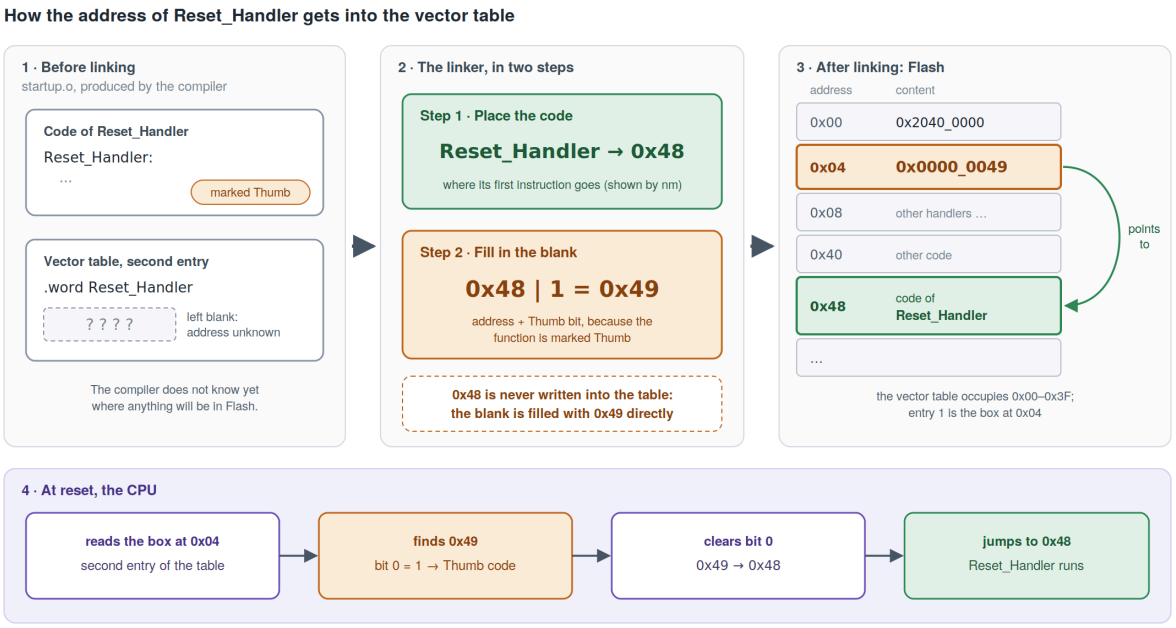
> **ONLY CODE POINTERS CARRY THE THUMB BIT THE THUMB BIT APPLIES ONLY TO ADDRESSES THE CPU JUMPS TO: FUNCTION ADDRESSES IN THE VECTOR TABLE, FUNCTION POINTERS, THE ELF ENTRY POINT. A DATA POINTER, SUCH AS `&COUNTER`, IS THE ADDRESS OF A VARIABLE THAT THE CPU READS OR WRITES, AND IS NEVER MODIFIED: IF `COUNTER` IS AT `0X20000004`, `&COUNTER` IS `0X20000004`.**
>
> The reason is alignment. Instructions always start at even addresses, so bit 0 of a code address is free to carry the Thumb bit. Data can also sit at odd addresses (a `char` at `0x20000005`), so for data bit 0 is a real part of the address.

2.2.1 How the linker sets the Thumb bit

The address of `Reset_Handler` reaches the vector table in four steps, from compilation to reset:

1. **Before linking**: `startup.o` contains the code of `Reset_Handler`, marked as Thumb. The second entry of the vector table only names `Reset_Handler` and is left **blank**, because its address is not known yet.

2. **Linking**: the linker first assigns the code of `Reset_Handler` the address `0x48`, which becomes the value of the symbol. It then resolves the reference in the vector table, filling the blank entry with `0x48 | 1 = 0x49`. The value `0x48` is never written into the table.
3. **After linking**: the entry at address `0x04` contains `0x49` and points to the code at `0x48`.
4. **At reset**: the CPU reads the entry at `0x04`, finds `0x49`, **clears bit 0**, and jumps to `0x48`.



2.2.1.1 WHICH TOOLS REPORT `0X48` AND WHICH `0X49`

Value	Meaning	Reported by
`0x48`	Location of the first instruction	Map file, `nm`, `objdump -d`
`0x49`	Code pointer, with the Thumb bit	`readelf -h` (entry point), `objdump -s -j .isr_vector` (vector table)

No instruction starts at `0x49`: the byte at `0x49` belongs to the instruction that starts at `0x48`, with **Thumb Bit** enabled.

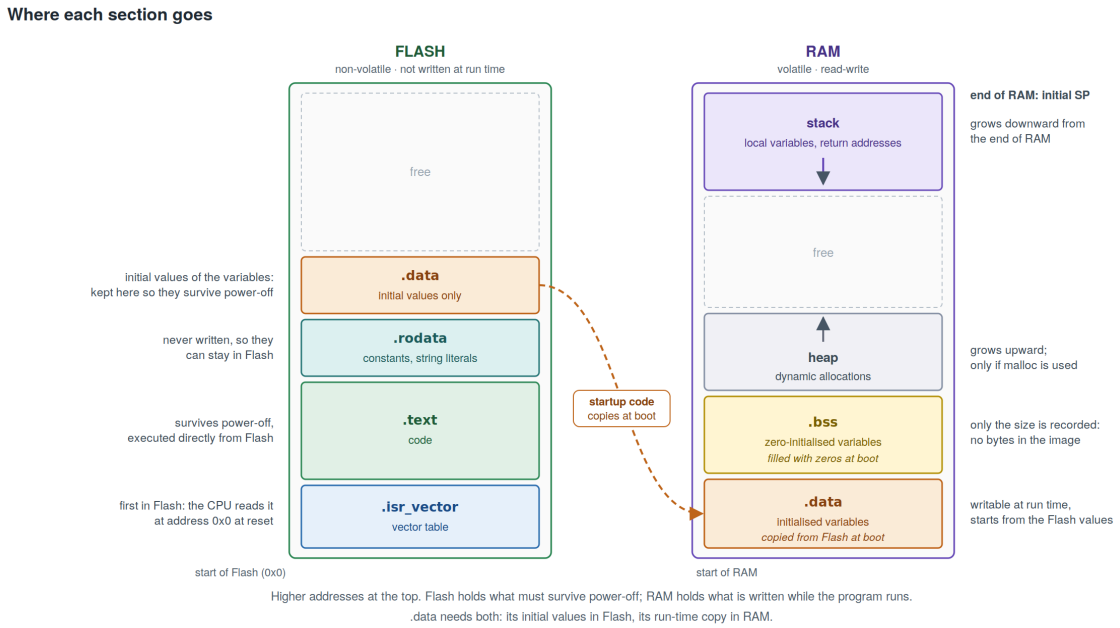
2.3 Memory Layout of the Image

The linker distributes the sections of the program between the memory regions declared in the linker script, Flash and RAM. Where each section goes depends on two questions: must its content survive power-off, and must it be writable at run time?

2.3.1 Where Each Section Goes

Section	Content	Placed in	Why
`.isr_vector`	Vector table	Start of Flash	The CPU reads it at address `0x0` at reset
`.text`	Code	Flash	Must survive power-off; executed directly from Flash
`.rodata`	Constants, string literals	Flash	Never written, so they can stay in Flash

Section	Content	Placed in	Why
`.data`	Initialised variables	RAM at run time, initial values stored in Flash	Must be writable, but their initial values must survive power-off: the startup code copies them
`.bss`	Zero-initialised variables	RAM, no bytes in the image	Only their size is recorded: the startup code fills them with zeros



RAM also holds two areas that contain nothing from the program's object files:

1. **heap**, space reserved by the linker script for dynamic allocation
2. the **stack**, which is not a section at all but an address range that grows downward from the end of RAM.

Inside each region the linker follows the **order of the linker script**, and inside each section the order of the object files on the command line. The order matters for the vector table: `.isr_vector` is written first in the script so that it starts at `0x0`, where the CPU looks for it.

The same layout gives the memory budgets:

```
Flash usage = .isr_vector + .text + .rodata + .data initial values
RAM usage   = .data + .bss + heap + stack
```

`.data` is counted twice, once in each budget. `arm-none-eabi-size` reports the totals as `text`, `data`, and `bss`; the size of the ELF file is not a measure of memory use, because it includes debug information and excludes `.bss`.

3 Inspecting the Image

The binutils tools **read the ELF file** on the host, without running anything on the target: they show what the toolchain produced, before the image is executed or debugged.

3.1 Inspection Commands

Command	Shows
`arm-none-eabi-size`	Section sizes in three totals (`text` , `data` , `bss`): the Flash and RAM budgets
`arm-none-eabi-nm`	Symbols with address and type letter (see Symbol Letters of nm)
`arm-none-eabi-readelf -h`	The ELF header, including the entry point
`arm-none-eabi-readelf -l`	The program headers: the segments loaded by QEMU
`arm-none-eabi-objdump -h`	One line per section: size, VMA, and LMA
`arm-none-eabi-objdump -d`	Disassembly: every instruction with the address where it is stored
`arm-none-eabi-objdump -s -j <section>`	The raw bytes of one section

RUN ADDRESS (VMA) AND LOAD ADDRESS (LMA)

In the `-h` listing each section has two addresses: the **VMA**, where the section is used at run time, and the **LMA**, where its bytes are stored in the image. They differ only for `.data` , whose initial values are stored in Flash and copied to RAM by the startup code.

The name "virtual memory address" comes from systems with an MMU. On a core without one, such as the Cortex-M3, both addresses are physical: the VMA is simply the run address.

READING AN `-s` DUMP

Each line shows the address of its first byte, 16 bytes in four groups of 4 in memory order, and the same bytes as text on the right (`.` for non-printable bytes).

3.2 Symbol Letters of nm

Letter	Meaning
`U`	Undefined: used here, defined in another file
`T` / `t`	Function (`.text`); `t` if `static`
`R` / `r`	Constant (`.rodata`); `r` if `static`
`D` / `d`	Initialised variable (`.data`); `d` if `static`
`B` / `b`	Zero-initialised variable (`.bss`); `b` if `static`

Letter	Meaning
`W`	Weak symbol, replaced by a normal definition with the same name
`A`	Absolute symbol, belonging to no section

3.3 Reading the Map File

The map file (`-Xlinker -Map=…`) is a text report written by the linker during the link. It records where each section and each symbol ended up, how big it is, and which object file it came from. It has two main parts: *Memory Configuration*, with the regions declared in the linker script, and *Linker script and memory map*, with the placement of every section.

Line type	Example	Meaning
Memory region	`FLASH 0x00000000 0x00400000`	Name, start, and length of a region
Output section (not indented)	`.text 0x00000040 0xa0`	A section of the final image: address and total size
Input section (indented)	`.text 0x00000040 0x64 build/boot/startup.o`	The piece of that section coming from one object file
Symbol	`0x00000048 Reset_Handler`	The final address of a function or variable

The map file answers questions such as: at which address did a function or variable end up, which object file contributed a piece of code, which section grew and made a region overflow. Static (file-local) symbols do not appear in it.

3.4 Which Tool Answers Which Question

Question	Tool
How much Flash and RAM does the program use?	`size`
At which address did a function or a variable end up?	`nm` , map file
Which object file supplied a piece of code?	map file
Why did a region overflow?	map file
What is the entry point?	`readelf -h`
Is the Thumb bit set in the vector table?	`objdump -s -j .isr_vector`
Is `.data` stored in Flash and run in RAM?	`objdump -h`

3.5 Working Example

This example follows a small program from source to memory: first it predicts in which section each declaration ends up, then it checks the prediction with `size` , `objdump` , the map file, and `nm` . The program is made of two source files, linked with a linker script `mps2_m3.ld` .

`src/main.c` declares one global of every kind:

```
#include <stdint.h>

const uint32_t limit = 100;
uint32_t counter = 5;
uint32_t total;
char buffer[64];
const char *greeting = "hello";

int main( void )
{
    total = counter + limit;
    buffer[0] = greeting[0];
    for ( ;; ) { }
}
```

`boot/startup.c` provides the vector table and a `Reset_Handler` that copies `.data` from Flash to RAM, fills `.bss` with zeros, and calls `main()`. The symbols `_estack`, `_sidata`, `_sdata`, `_edata`, `_sbss`, and `_ebss` are not variables: the linker script defines them as **addresses**, so the code uses only their address (`&_sdata`).

```
#include <stdint.h>

extern uint32_t _estack, _sidata, _sdata, _edata, _sbss, _ebss;
int main( void );

void Default_Handler( void )
{
    for ( ;; ) { }
}

void Reset_Handler( void )
{
    uint32_t *src = &_sidata;
    uint32_t *dst = &_sdata;
    while ( dst < &_edata )          /* copy .data from Flash to RAM */
        *dst++ = *src++;
    for ( dst = &_sbss; dst < &_ebss; )
        *dst++ = 0;                  /* zero .bss */
    main();
    for ( ;; ) { }
}

__attribute__(( section( ".isr_vector" ), used ))
```

```
const uint32_t vector_table[16] = {
    (uint32_t) &_estack,             /* 0: initial stack pointer */
    (uint32_t) Reset_Handler,        /* 1: reset                 */
    (uint32_t) Default_Handler,      /* 2: NMI                   */
    (uint32_t) Default_Handler,      /* 3: HardFault             */
};                                   /* 4-15: left at 0          */
```

`scripts/mps2_m3.ld` describes the two memory regions and the order of the output sections:

```
MEMORY
{
    FLASH (rx)  : ORIGIN = 0x00000000, LENGTH = 4M
    RAM   (rwx) : ORIGIN = 0x20000000, LENGTH = 4M
}

_estack = ORIGIN(RAM) + LENGTH(RAM);

ENTRY(Reset_Handler)

SECTIONS
{
    .isr_vector :
    {
        . = ALIGN(4);
        KEEP(*(.isr_vector))
    } > FLASH

    .text :
    {
        *(.text*)
        *(.rodata*)
        . = ALIGN(4);
        _etext = .;
    } > FLASH

    _sidata = LOADADDR(.data);
    .data :
    {
        . = ALIGN(4);
        _sdata = .;
        *(.data*)
        . = ALIGN(4);
        _edata = .;
```

```
    } > RAM AT > FLASH

    .bss (NOLOAD) :
    {
        . = ALIGN(4);
        _sbss = .;
        *(.bss*)
        *(COMMON)
        . = ALIGN(4);
        _ebss = .;
    } > RAM
}
```

The image is built with these commands:

```
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -ffreestanding -std=gnu17 -O0 -g3 -Wall \
    -c boot/startup.c -o build/startup.o
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -ffreestanding -std=gnu17 -O0 -g3 -Wall \
    -c src/main.c -o build/main.o
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -nostdlib \
    -T scripts/mps2_m3.ld -Xlinker -Map=build/output.map \
    build/startup.o build/main.o -lgcc -o build/example.axf
```

3.5.1 Step 1: Predict

Declaration	Section	Region
`main`, `Reset_Handler`	`.text`	Flash
`const uint32_t limit = 100;`	`.rodata`	Flash
`uint32_t counter = 5;`	`.data`	RAM, initial value stored in Flash
`uint32_t total;`, `char buffer[64];`	`.bss`	RAM, zeroed at boot
`const char *greeting = "hello";`	The pointer in `.data`, the characters in `.rodata`	Pointer in RAM, string in Flash

The last row is the subtle one: `greeting` is a writable variable that holds an address, while the string it points to is a constant.

3.5.2 Step 2: Measure the budgets with `size`

```
$ arm-none-eabi-size build/example.axf
   text    data     bss     dec     hex filename
```

```
    224       8      68     300     12c build/example.axf
```

- `text` = 224 bytes: the 64-byte vector table plus 160 bytes of code and constants.
- `data` = 8 bytes: `counter` and `greeting`, one 4-byte word each.
- `bss` = 68 bytes: `total` (4) and `buffer` (64).

Flash holds `text + data` = 232 bytes; RAM holds `data + bss` = 76 bytes, plus the stack. The ELF file itself is about 29 KB, almost all debug information.

3.5.3 Step 3: Check the sections with `objdump -h`

```
$ arm-none-eabi-objdump -h build/example.axf
Idx Name          Size      VMA       LMA
  0 .isr_vector   00000040  00000000  00000000
  1 .text         000000a0  00000040  00000040
  2 .data         00000008  20000000  000000e0
  3 .bss          00000044  20000008  000000e8
```

The vector table starts at `0x0` and `.text` follows it. `.data` runs at `0x20000000`, the start of RAM, but its bytes are stored at `0xE0`, right after the code: this is where the startup code copies them from. `.bss` follows `.data` in RAM; it has no bytes in the image (`objdump` marks it `ALLOC` only), so its LMA has no meaning. The debug sections listed after these are omitted.

3.5.4 Step 4: Read the map file

```
Memory Configuration

Name             Origin             Length             Attributes
FLASH            0x00000000         0x00400000         xr
RAM              0x20000000         0x00400000         xrw

Linker script and memory map

.isr_vector     0x00000000       0x40
 .isr_vector    0x00000000       0x40 build/startup.o
                0x00000000                vector_table

.text           0x00000040       0xa0
 .text          0x00000040       0x64 build/startup.o
                0x00000040                Default_Handler
                0x00000048                Reset_Handler
 .text          0x000000a4       0x30 build/main.o
                0x000000a4                main
 .rodata        0x000000d4        0xa build/main.o
```

```
                0x000000d4                limit
 *fill*         0x000000de        0x2
                0x000000e0                        _sidata = LOADADDR (.data)

.data           0x20000000        0x8 load address 0x000000e0
 .data          0x20000000        0x8 build/main.o
                0x20000000                counter
                0x20000004                greeting

.bss            0x20000008       0x44
 .bss           0x20000008       0x44 build/main.o
                0x20000008                total
                0x2000000c                buffer
```

The excerpt keeps only the lines discussed here. It confirms the prediction piece by piece: the code of each object file inside `.text`, the constants of `main.o` (`limit` and the 6 bytes of `"hello"`, 10 bytes in total) in a `.rodata` piece inside `.text`, two bytes of `*fill*` to end `.text` on a multiple of 4, and the `load address` of `.data`, which is also the value of `_sidata`, the symbol the startup code uses as the source of the copy.

3.5.5 Step 5: Find every symbol with `nm`

```
$ arm-none-eabi-nm -n build/example.axf
00000000 R vector_table
00000040 T Default_Handler
00000048 T Reset_Handler
000000a4 T main
000000d4 T limit
000000e0 T _etext
000000e0 A _sidata
20000000 D _sdata
20000000 D counter
20000004 D greeting
20000008 D _edata
20000008 B _sbss
20000008 B total
2000000c B buffer
2000004c B _ebss
20400000 R _estack
```

`counter` and `greeting` are `D`, `total` and `buffer` are `B`, as predicted. `limit` is the surprise: in `main.o` it is `R`, but in the final image it is `T`, because the linker script folds `.rodata` into the `.text` output section and `nm` classifies a symbol by the output section it ended up in. The string `"hello"` has no name, so it does not appear.

3.5.6 Step 6: Read the bytes with `objdump -s`

```
$ arm-none-eabi-objdump -s -j .isr_vector build/example.axf
 0000 00004020 49000000 41000000 41000000  ..@ I...A...A...

$ arm-none-eabi-objdump -s -j .text build/example.axf
 00d0 0c000020 64000000 68656c6c 6f000000  ... d...hello...

$ arm-none-eabi-objdump -s -j .data build/example.axf
 20000000 05000000 d8000000                    ........
```

The Cortex-M3 is little-endian: a 32-bit word is stored lowest byte first, so `00004020` is `0x20400000`.

- **Vector table**: word 0 is `0x20400000`, the initial stack pointer (`_estack`); word 1 is `0x49`, the address of `Reset_Handler` with the Thumb bit; words 2 and 3 are `0x41`, the address of `Default_Handler` with the Thumb bit.
- **Constants**: at `0xD4`, `64000000` is `limit` = 100; at `0xD8` the characters `hello` followed by the terminating zero.
- **Initial values of `.data`**: `05000000` is `counter` = 5, and `d8000000` is `greeting` = `0xD8`, exactly the Flash address of `"hello"`. A pointer is nothing more than a variable whose value is an address.

3.5.7 Result

Flash and RAM of the example



4 Running and Debugging the Image

4.1 Running the Image in QEMU

QEMU **emulates** the target board on the host, so the image can run before any physical hardware is used:

```
qemu-system-arm -machine mps2-an385 -kernel build/scratch-os.axf -nographic -semihosting
```

Option	Meaning
`-machine mps2-an385`	The board to emulate (here a Cortex-M3): its memory map must match the linker script
`-kernel`	The ELF image to boot
`-nographic`	Routes the target console to the host terminal
`-semihosting`	Lets the program print through the host (see below)

> ⚠ **QEMU DOES NOT EXIT**
>
> A bare-metal program has no operating system to return to: when `main()` returns, the startup code enters an infinite loop, so QEMU keeps running and must be stopped by hand.

4.2 Semihosting: Output Through the Host

A bare-metal program has **no console**: `printf` needs a C library and an operating system, and a UART needs a driver. **Semihosting** solves this during **development**: the program asks the **host**, the computer running QEMU or a debugger, to perform the output on its behalf.

The request follows a convention defined by ARM:

Element	Role	For printing a string
`r0`	Which service is requested	`0x04` (`SYS_WRITE0`)
`r1`	The argument of the service	Address of the string
`bkpt 0xab`	Signals that the request is ready	—

When the CPU executes `bkpt 0xab`, QEMU stops it, reads `r0` and `r1`, prints the string on the host terminal, and resumes the program at the next instruction. This is a **trap**: control passes to a more powerful entity, which performs the service and returns. A system call has the same shape, with the kernel in the role of the host.

```
static void semihost_write0( const char *message )
{
    register uint32_t    r0 __asm__( "r0" ) = 0x04;    /* SYS_WRITE0 */
    register const char *r1 __asm__( "r1" ) = message; /* string address */
    __asm__ volatile ( "bkpt 0xab" : : "r"( r0 ), "r"( r1 ) : "memory" );
}
```

C cannot choose registers nor emit `bkpt`, so the function uses two GCC extensions: `register … __asm__( "r0" )` forces a variable into a register, and inline assembly inserts the instruction (`volatile` stops the compiler from removing it).

> ⚠ **DEVELOPMENT TOOL ONLY**
>
> Semihosting works only when QEMU or a debugger is listening. On a real board with nothing attached, `bkpt` simply halts the CPU: production firmware prints through a peripheral such as a UART.

4.3 Debugging with `gdb`

A **debugger** stops the program, executes it one instruction or one line at a time, and shows registers and memory. On a microcontroller, where a crash often prints nothing, it is often the only way to see what the CPU does.

Debugging is **remote**: the program runs in QEMU, started with two extra options, and **gdb connects to it over TCP**:

- `-S`: freeze the CPU before its first instruction;
- `-gdb tcp::1234`: wait for gdb on port 1234.

gdb (`arm-none-eabi-gdb`, or `gdb-multiarch` on Linux) is started on the same ELF, from which it reads only symbols and debug information.

Command	Effect
`target remote :1234`	Connect to QEMU
`info registers sp pc`	Show registers
`x/2wx 0`	Examine 2 words in hex at address 0
`p/x $r1`	Print a register or an expression in hex
`stepi` (`si`)	Execute one machine instruction
`break main` (`b`)	Stop when execution reaches `main`
`continue` (`c`)	Run to the next breakpoint

4.4 Locating a Failure

Each kind of failure points to a **different stage** of the pipeline:

Symptom	Where to look
The compiler rejects a command	Toolchain installation and Makefile flags
Unresolved symbol	Object files on the link command and matching declarations
Section overflow	Memory regions in the linker script, map file

Symptom	Where to look
QEMU runs but prints nothing	Reset path, output call, `-semihosting` option; attach gdb to see how far execution gets

LOCATE THE FAILING STAGE

Read the **first** diagnostic and identify which stage failed before changing anything: changing compiler flags at random is not a diagnosis.

5 Build Automation with Make

`make` is not a compiler: it reads a `Makefile`, which describes how each file of the build is produced, and runs only the commands needed to bring the requested files up to date.

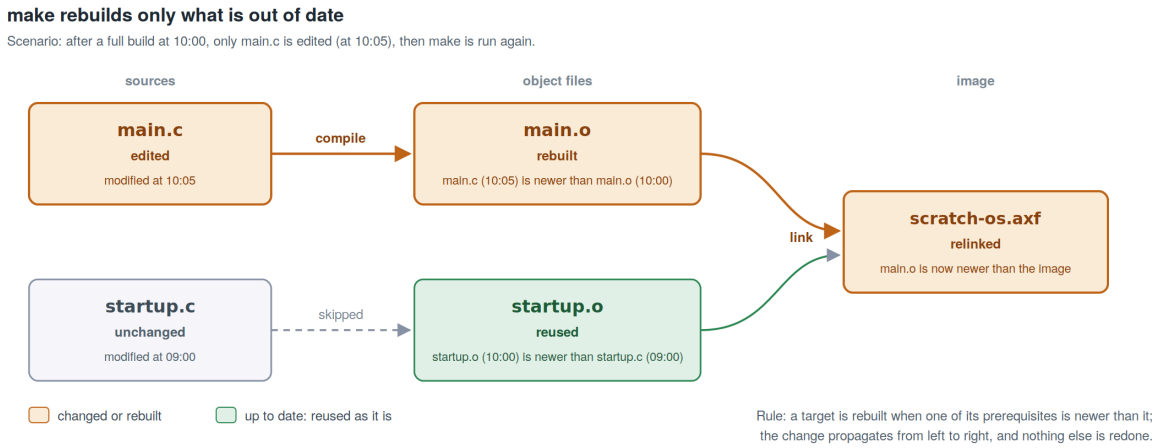
5.1 Rules and Rebuilds

A `Makefile` is a list of **rules**:

```
target: prerequisite1 prerequisite2
        recipe line 1
        recipe line 2
```

Part	Meaning
`target`	The file to produce, such as an object file or the ELF image
`prerequisites`	The files the target is made from; `make` brings them up to date first
`recipe`	The shell commands that produce the target; every line **starts with a Tab**, not spaces

A target is rebuilt only if it does not exist or if one of its prerequisites is **newer** than it (by modification time). Since prerequisites can be targets of other rules, the rules form a dependency graph, and a change propagates along it:



The rebuild is correct only if the graph lists every real dependency. A header or a linker script that is not listed as a prerequisite does not trigger a rebuild when it changes: `make clean && make` forces a full one.

5.2 The Tab Rule

A recipe line indented with spaces stops `make` with an error that names the line but not the cause:

```
$ make
Makefile:5: *** missing separator.  Stop.
```

Spaces and a Tab look the same in an editor; `cat -et` shows the difference by printing each Tab as `^I` and each line end as `$`:

```
$ cat -et Makefile
$(BUILD_DIR)/%.o: %.c$
    @mkdir -p $(dir $@)$
^I$(CC) $(CFLAGS) -c $< -o $@$
```

The first recipe line starts with four spaces, the second with a Tab (`^I`). Many editors insert spaces when the Tab key is pressed: configure them to keep real Tabs in Makefiles.

5.3 Variables

```
CC      := arm-none-eabi-gcc
CFLAGS  := -mthumb -mcpu=cortex-m3
CFLAGS  += -Wall -Wextra
```

Operator	Behaviour
`:=`	Evaluates the right-hand side once, immediately: the usual choice
`=`	Re-evaluates the right-hand side at every use
`+=`	Appends to the current value
`?=`	Assigns only if the variable is not already set, so it can be overridden from the command line (`make CC=clang`)

A variable is used with `$(NAME)` . By convention, `CC` , `CFLAGS` , `LDFLAGS` , and `LDLIBS` hold the compiler and its options; other names, such as `SOURCES` or `BUILD_DIR` , are the Makefile's own.

5.4 Pattern Rules and Automatic Variables

One **pattern rule** compiles every source file, instead of one rule per file:

```
$(BUILD_DIR)/%.o: %.c
        @mkdir -p $(dir $@)
        $(CC) $(CFLAGS) -c $< -o $@
```

`%` matches the same text on both sides: for `build/src/main.o` it matches `src/main` , so the prerequisite is `src/main.c` . Inside the recipe, **automatic variables** name the files of the rule being run:

Variable	Meaning
`$@`	The target
`$<`	The first prerequisite
`$^`	All the prerequisites
`$(dir $@)`	The directory of the target, created with `mkdir -p` before the compiler writes into it

The list of objects is derived from the list of sources with a **substitution**: `$(SOURCES:%.c=$(BUILD_DIR)/%.o)`

`` turns `src/main.c` into `build/src/main.o`. The source list can be written by hand or collected with `$(wildcard src/*.c)`: the explicit list is easier to check, the wildcard picks up new files automatically.

5.5 Phony Targets

```
.PHONY: all run clean
```

`run` and `clean` are actions, not files. Without `.PHONY`, a file called `clean` in the directory would make `make` think the target is up to date and skip it. `all`, the first target of the file, is the **default goal**, built when `make` is run with no arguments.

5.6 Debugging a Makefile

A Makefile can be wrong without failing, for example when a substitution builds the wrong list. What matters is what the variables expand to:

Tool	Effect
`$(info OBJECTS = $(OBJECTS))`	Prints a value while the Makefile is read
`$(warning …)`	Same, with the `file:line:` of the call
`make -n`	Prints the commands it would run, without running them

5.7 A Complete Makefile

```
CC       := arm-none-eabi-gcc
CFLAGS   := -mthumb -mcpu=cortex-m3 -nostartfiles -ffreestanding -std=gnu17
CFLAGS   += -Wall -Wextra -O0 -g3
LDFLAGS  := -T scripts/mps2_m3.ld
LDFLAGS  += -Xlinker -Map=build/output.map
LDFLAGS  += -nostdlib -lgcc

BUILD_DIR := build
BIN       := $(BUILD_DIR)/scratch-os.axf
SOURCES   := boot/startup.c src/main.c
OBJECTS   := $(SOURCES:%.c=$(BUILD_DIR)/%.o)

.PHONY: all run clean
all: $(BIN)

$(BUILD_DIR)/%.o: %.c
        @mkdir -p $(dir $@)
        $(CC) $(CFLAGS) -c $< -o $@
```

```
$(BIN): $(OBJECTS)
        $(CC) $(CFLAGS) $(LDFLAGS) $(OBJECTS) -o $(BIN)

run: $(BIN)
        qemu-system-arm -machine mps2-an385 -kernel $(BIN) -nographic -semihosting

clean:
        rm -rf $(BUILD_DIR)
```

`make` compiles the stale objects and links them; run again with no changes, it does nothing. `make run` also boots the image in QEMU, and `make clean` deletes the `build/` directory, which holds every generated file.

> ⚠ **LIBRARY ORDER**
>
> The linker takes from a library only the functions requested by the objects read **before** it. With `-lgcc` in `LDFLAGS`, which comes before the objects on the command line, code that needs a libgcc helper (such as a 64-bit division, `__aeabi_uldivmod`) fails to link. Moving `-lgcc` after the objects fixes it.